

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Богатырева Е.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.09.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создаёт дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процессы представлены разными программами. Родительский процесс передаёт команды пользователя через `pipe1`, который связан со стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передаёт данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл.

Пользователь вводит команды вида: «число число число<newline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `float`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)` — создаёт дочерний процесс; после вызова код выполняется в двух процессах параллельно.
- `int pipe(int pipefd[2])` — создаёт неименованный канал; `pipefd[0]` — для чтения, `pipefd[1]` — для записи. Использован дважды: `pipe1` (родитель → ребёнок) и `pipe2` (ребёнок → родитель).
- `int dup2(int oldfd, int newfd)` — переназначает файловый дескриптор; в дочернем процессе `pipe1[0]` подключается к `STDIN_FILENO`, а `pipe2[1]` — к `STDOUT_FILENO`.
- `int execve(const char *pathname, char *const argv[], char *const envp[])` — заменяет образ памяти процесса; в дочернем процессе запускает программу `./child`, передавая ей имя файла как аргумент `argv[1]`.
- `int open(const char *pathname, int flags, mode_t mode)` — открывает файл; в дочернем процессе открывает целевой файл с флагами `O_WRONLY | O_CREAT | O_TRUNC`.
- `ssize_t read(int fd, void *buf, size_t count)` — читает данные из дескриптора (используется для чтения из `pipe` и `stdin`).
- `ssize_t write(int fd, const void *buf, size_t count)` — пишет данные в дескриптор (используется для записи в `pipe`, `stdout`, `stderr` и целевой файл).
- `int close(int fd)` — закрывает файловый дескриптор.
- `pid_t waitpid(pid_t pid, int *wstatus, int options)` — ожидает завершения дочернего процесса и получает его статус.
- `void _exit(int status)` — немедленно завершает процесс без сброса буферов `stdio` (используется в дочернем процессе при ошибках до `execve`).

Алгоритм работы программы:

Программа состоит из двух независимых исполняемых файлов — `parent` (родительский процесс) и `child` (дочерний процесс).

1. **Родительский процесс** запрашивает у пользователя имя файла и читает его со стандартного ввода. Имя файла сохраняется для передачи дочернему процессу.
2. Родитель создаёт два неименованных канала: `pipe1` для передачи данных от родителя к ребёнку и `pipe2` для передачи данных от ребёнка к родителю.
3. С помощью `fork()` создаётся дочерний процесс.
4. **В дочернем процессе:**
 - закрываются неиспользуемые концы каналов;
 - `pipe1[0]` дублируется в `STDIN_FILENO` через `dup2` — теперь стандартный ввод ребёнка связан с каналом `pipe1`;
 - `pipe2[1]` дублируется в `STDOUT_FILENO` — стандартный вывод ребёнка связан с каналом `pipe2`;
 - через `execve()` запускается программа `./child`, которой имя файла передаётся как `argv[1]`.
5. **В родительском процессе:**
 - закрываются неиспользуемые концы каналов;
 - в цикле пользователю предлагается вводить команды — строки с числами. Каждая введённая строка целиком записывается в `pipe1[1]` и, следовательно, попадает на стандартный вход дочернего процесса;
 - пустая строка (или `Ctrl+D`) означает конец ввода — родитель закрывает `pipe1[1]`, посылая ребёнку сигнал EOF;
 - затем родитель читает из `pipe2[0]` сообщения от ребёнка и выводит их на экран;
 - в конце через `waitpid()` дожидается завершения ребёнка и печатает его код возврата.
6. **Дочерний процесс (после `execve`):**
 - открывает файл с именем `argv[1]` на запись (`O_WRONLY | O_CREAT | O_TRUNC`);
 - в цикле читает строки со стандартного ввода (то есть из `pipe1`);
 - для каждой строки разбирает все числа типа `float` с помощью собственного парсера (поддерживает знак, десятичную точку и экспоненту);
 - суммирует числа и формирует строку результата в формате `%f` (6 знаков после запятой) собственной функцией форматирования;
 - записывает полученную строку в открытый файл;
 - по достижении EOF закрывает файл и отправляет родителю через `pipe2` (свой `stdout`) сообщение об успешном завершении.

Код программы

parent.c

```
#include <unistd.h>

#include <sys/wait.h>

#include <string.h>

static void put_str(int fd, const char *s) {

    size_t len = strlen(s);

    size_t off = 0;

    while (off < len) {

        ssize_t n = write(fd, s + off, len - off);

        if (n <= 0) return;

        off += (size_t)n;

    }

}

static ssize_t read_line(int fd, char *buf, size_t maxlen) {

    size_t i = 0;

    while (i < maxlen - 1) {

        char c;

        ssize_t n = read(fd, &c, 1);

        if (n < 0) {

            if (i == 0) return -1;

            break;

        }

        if (n == 0) {

            if (i == 0) return 0;

            break;

        }

        buf[i++] = c;

        if (c == '\n') break;

    }

    buf[i] = '\0';

    return (ssize_t)i;

}

static void put_int(int fd, int v) {
```

```

char buf[16];

int pos = 0;

if (v == 0) {
    buf[pos++] = '0';
} else {
    if (v < 0) { buf[pos++] = '-'; v = -v; }

    char tmp[16];

    int t = 0;

    while (v > 0) { tmp[t++] = (char)('0' + (v % 10)); v /= 10; }

    while (t > 0) buf[pos++] = tmp[--t];
}

write(fd, buf, (size_t)pos);
}

int main(void) {
    char fileName[1024];

    put_str(STDOUT_FILENO, "Enter file name: ");

    ssize_t r1 = read_line(STDIN_FILENO, fileName, sizeof(fileName));

    if (r1 <= 0) {
        put_str(STDERR_FILENO, "Error: cannot read file name\n");

        return 1;
    }

    size_t len = strlen(fileName);

    if (len > 0 && fileName[len - 1] == '\n') fileName[len - 1] = '\0';

    if (strlen(fileName) == 0) {
        put_str(STDERR_FILENO, "Error: empty file name\n");

        return 1;
    }

    int pipe1[2];

    int pipe2[2];

    if (pipe(pipe1) == -1) {
        put_str(STDERR_FILENO, "Error: pipe1 creation failed\n");

        return 1;
    }

    if (pipe(pipe2) == -1) {
        put_str(STDERR_FILENO, "Error: pipe2 creation failed\n");

```

```

        close(pipe1[0]); close(pipe1[1]);

        return 1;
    }

    pid_t pid = fork();

    if (pid == -1) {
        put_str(STDERR_FILENO, "Error: fork failed\n");

        close(pipe1[0]); close(pipe1[1]);

        close(pipe2[0]); close(pipe2[1]);

        return 1;
    }

    if (pid == 0) {
        close(pipe1[1]);

        close(pipe2[0]);

        if (dup2(pipe1[0], STDIN_FILENO) == -1) {
            put_str(STDERR_FILENO, "Error: dup2(stdin) failed\n");
            _exit(1);
        }

        close(pipe1[0]);

        if (dup2(pipe2[1], STDOUT_FILENO) == -1) {
            put_str(STDERR_FILENO, "Error: dup2(stdout) failed\n");
            _exit(1);
        }

        close(pipe2[1]);

        char *argv[] = { (char*)"./child", fileName, NULL };
        char *envp[] = { NULL };

        execve("./child", argv, envp);

        put_str(STDERR_FILENO, "Error: execve failed\n");
        _exit(1);
    }

    close(pipe1[0]);

    close(pipe2[1]);

    put_str(STDOUT_FILENO,

        "Enter numbers (e.g. '1.5 2.3 3.7'), empty line to exit:\n");

    char line[4096];

    while (1) {

```

```

put_str(STDOUT_FILENO, "> ");

ssize_t n = read_line(STDIN_FILENO, line, sizeof(line));

if (n <= 0) break;

int isEmpty = 1;

for (ssize_t i = 0; i < n; i++) {
    if (line[i] != ' ' && line[i] != '\t' && line[i] != '\n') {
        isEmpty = 0;
        break;
    }
}

if (isEmpty) break;

ssize_t written = 0;

while (written < n) {
    ssize_t w = write(pipe1[1], line + written, (size_t)(n - written));

    if (w <= 0) {
        put_str(STDERR_FILENO, "Error: write to pipe1 failed\n");
        close(pipe1[1]);
        close(pipe2[0]);
        waitpid(pid, NULL, 0);
        return 1;
    }

    written += w;
}

}

close(pipe1[1]);

char buf[4096];

ssize_t n;

while ((n = read(pipe2[0], buf, sizeof(buf))) > 0) {
    write(STDOUT_FILENO, buf, (size_t)n);
}

close(pipe2[0]);

int status;

if (waitpid(pid, &status, 0) == -1) {
    put_str(STDERR_FILENO, "Error: waitpid failed\n");
    return 1;
}

```

```

    }

    if (WIFEXITED(status)) {
        put_str(STDOUT_FILENO, "Child exited with code ");
        put_int(STDOUT_FILENO, WEXITSTATUS(status));
        put_str(STDOUT_FILENO, "\n");
    } else if (WIFSIGNALED(status)) {
        put_str(STDOUT_FILENO, "Child killed by signal ");
        put_int(STDOUT_FILENO, WTERMSIG(status));
        put_str(STDOUT_FILENO, "\n");
    }

    return 0;
}

```

child.c

```

#include <unistd.h>

#include <fcntl.h>

#include <string.h>

static void put_str(int fd, const char *s) {
    size_t len = strlen(s);
    size_t off = 0;
    while (off < len) {
        ssize_t n = write(fd, s + off, len - off);
        if (n <= 0) return;
        off += (size_t)n;
    }
}

static ssize_t read_line(int fd, char *buf, size_t maxlen) {
    size_t i = 0;
    while (i < maxlen - 1) {
        char c;
        ssize_t n = read(fd, &c, 1);
        if (n < 0) {
            if (i == 0) return -1;
            break;
        }
    }
}

```

```

        if (n == 0) {
            if (i == 0) return 0;

            break;
        }

        buf[i++] = c;

        if (c == '\n') break;
    }

    buf[i] = '\0';

    return (ssize_t)i;
}

static int parse_float(const char **p, float *out) {
    const char *s = *p;

    while (*s == ' ' || *s == '\t') s++;

    int sign = 1;

    if (*s == '-') { sign = -1; s++; }
    else if (*s == '+') { s++; }

    if (*s < '0' || *s > '9') return 0;

    float val = 0.0f;

    while (*s >= '0' && *s <= '9') {
        val = val * 10.0f + (float)(*s - '0');

        s++;
    }

    if (*s == '.') {
        s++;

        float frac = 0.1f;

        while (*s >= '0' && *s <= '9') {
            val += (float)(*s - '0') * frac;

            frac *= 0.1f;

            s++;
        }
    }

    if (*s == 'e' || *s == 'E') {
        const char *save = s;

        s++;

        int esign = 1;

```

```

    if (*s == '-') { esign = -1; s++; }

    else if (*s == '+') { s++; }

    if (*s >= '0' && *s <= '9') {

        int exp = 0;

        while (*s >= '0' && *s <= '9') {

            exp = exp * 10 + (*s - '0');

            s++;

        }

        float mult = 1.0f;

        for (int i = 0; i < exp; i++) mult *= 10.0f;

        if (esign < 0) val /= mult;

        else val *= mult;

    } else {

        s = save;

    }

}

*out = sign * val;

*p = s;

return 1;

}

static int float_to_str(float v, char *out, size_t outsz) {

    size_t pos = 0;

    if (outsz < 16) return -1;

    if (v < 0) {

        out[pos++] = '-';

        v = -v;

    }

    long long scaled = (long long)((double)v * 1000000.0 + 0.5);

    long long intPart = scaled / 1000000LL;

    long long fracPart = scaled % 1000000LL;

    char tmp[32];

    int t = 0;

    if (intPart == 0) {

        tmp[t++] = '0';

    } else {

```

```

        while (intPart > 0) {
            tmp[t++] = (char)('0' + (intPart % 10));
            intPart /= 10;
        }
    }
    while (t > 0) out[pos++] = tmp[--t];
    out[pos++] = '.';
    char frac[6];
    for (int i = 5; i >= 0; i--) {
        frac[i] = (char)('0' + (fracPart % 10));
        fracPart /= 10;
    }
    for (int i = 0; i < 6; i++) out[pos++] = frac[i];
    out[pos++] = '\n';
    out[pos] = '\0';
    return (int)pos;
}

int main(int argc, char **argv) {
    if (argc < 2) {
        put_str(STDERR_FILENO, "Error: file name argument required\n");
        return 1;
    }
    const char *fileName = argv[1];
    int fd = open(fileName, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        put_str(STDERR_FILENO, "Error: cannot open file for writing\n");
        return 1;
    }
    char line[4096];
    while (1) {
        ssize_t n = read_line(STDIN_FILENO, line, sizeof(line));
        if (n <= 0) break;
        const char *p = line;
        float sum = 0.0f;
        int count = 0;

```

```

while (*p) {
    while (*p == ' ' || *p == '\t' || *p == '\n' || *p == '\r') p++;
    if (*p == '\0') break;
    float val;
    if (!parse_float(&p, &val)) {
        p++;
        continue;
    }
    sum += val;
    count++;
}
if (count == 0) continue;
char outbuf[64];
int m = float_to_str(sum, outbuf, sizeof(outbuf));
if (m > 0) {
    ssize_t off = 0;
    while (off < m) {
        ssize_t w = write(fd, outbuf + off, (size_t)(m - off));
        if (w <= 0) break;
        off += w;
    }
}
}
close(fd);
put_str(STDOUT_FILENO, "Child: done, results written to file\n");
return 0;
}

```

Протокол работы программы

Тестирование:

1. \$./parent

Enter file name: result.txt

Enter numbers (e.g. '1.5 2.3 3.7'), empty line to exit:

> 1.5 2.5 3.0

> 10.25 20.75

>

Child: done, results written to file

Child exited with code 0

Результат:

\$ cat result.txt

7.000000

31.000000

2. \$./parent

Enter file name: test.txt

Enter numbers (e.g. '1.5 2.3 3.7'), empty line to exit:

> 1

> 8.5

> -3.9

> 4.0

>

Child: done, results written to file

Child exited with code 0

Результат:

\$ cat test.txt

1.000000

8.500000

-3.900000

4.000000

3. \$./parent

Enter file name: test3.txt

Enter numbers (e.g. '1.5 2.3 3.7'), empty line to exit:

> abc 4.5 def

> 1 2 3 end

>

Child: done, results written to file

Child exited with code 0

Результат:

\$ cat test3.txt

4.500000

6.000000

Вывод

В ходе выполнения лабораторной работы были получены практические навыки управления процессами в ОС Linux и организации межпроцессного взаимодействия через неименованные каналы (pipe). Разработаны две самостоятельные программы на языке Си — parent и child, — реализующие схему взаимодействия «родитель → ребёнок → файл» с использованием системных вызовов. Родительский процесс передаёт введённые пользователем строки через pipe1 на стандартный ввод дочернего процесса, а дочерний процесс считает сумму чисел типа float в каждой строке, записывает результат в файл и уведомляет родителя о завершении через pipe2.